Analyse Détaillée : Evolution du diagramme de structure du programme

A. Changement de Paradigme : Disparition du Contrôleur (MVC → Event-Driven)

- **Initial (Théorie) :** Le diagramme initial suivait un pattern **MVC (Modèle-Vue-Contrôleur)** strict. Un fichier central `app_controller.c` devait servir de "chef d'orchestre", interceptant toutes les actions de l'IHM pour appeler les services.
- **Final (Réalité) :** Le rôle du contrôleur a été fusionné directement dans **`ihm.c`**.
- **Justification :** En développement GTK, il est souvent plus pragmatique de lier les "signaux" (clics boutons) directement aux fonctions logiques. Maintenir un contrôleur séparé aurait ajouté une complexité inutile (trop de fonctions intermédiaires) pour un projet de cette taille. L'architecture est devenue **"Event-Driven"** (pilotée par les événements).

B. Optimisation du Flux d'Importation

- **Initial :** Le diagramme suggérait que le Contrôleur récupérait les données scannées par l'`importer`, puis les renvoyait au DAO pour stockage.
- **Final :** Le module **`importer.c`** communique directement avec **`dao_musique.c`**.
- **Justification :** Performance et simplicité. Au lieu de faire transiter des milliers de structures de données par l'interface ou un contrôleur (ce qui consomme de la mémoire), l'importateur insère les données en base au fur et à mesure qu'il scanne les fichiers. C'est un couplage plus fort, mais beaucoup plus efficace.

C. Simplification des Utilitaires (KISS - Keep It Simple, Stupid)

- **Initial :** Présence de modules "transverses" comme `logger.c` (gestion des logs) et `utils.c` ou `errors.h`.
- **Final :** Ces modules ont été supprimés.
- **Justification :** L'équipe a privilégié l'utilisation des fonctions standards (`printf`, `fprintf`) pour le débogage et la gestion d'erreurs directe, afin de se concentrer sur les fonctionnalités métier critiques (Lecture, BDD) plutôt que sur l'infrastructure.

D. Périmètre Fonctionnel (Feature Scope)

- **Initial :** Présence d'un module **`karaoke.c`**.
- **Final :** Ce module est absent du code actif, bien que la structure de données (`dao_paroles.c` et table SQL) ait été préparée.
- **Justification :** Gestion des priorités. L'intégration de l'affichage synchronisé a été dépriorisée pour garantir la stabilité de la lecture audio et de la gestion de base de données.
-

Diagramme de structure du programme Initial :

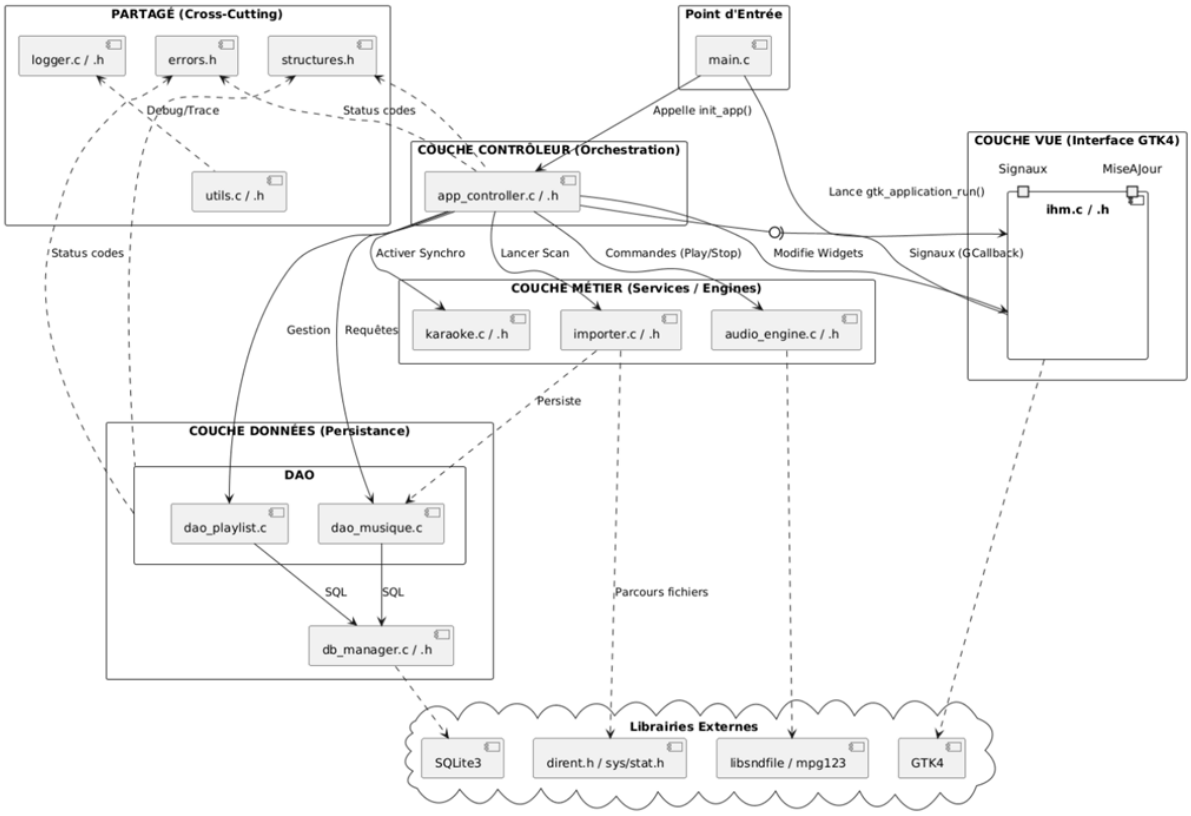
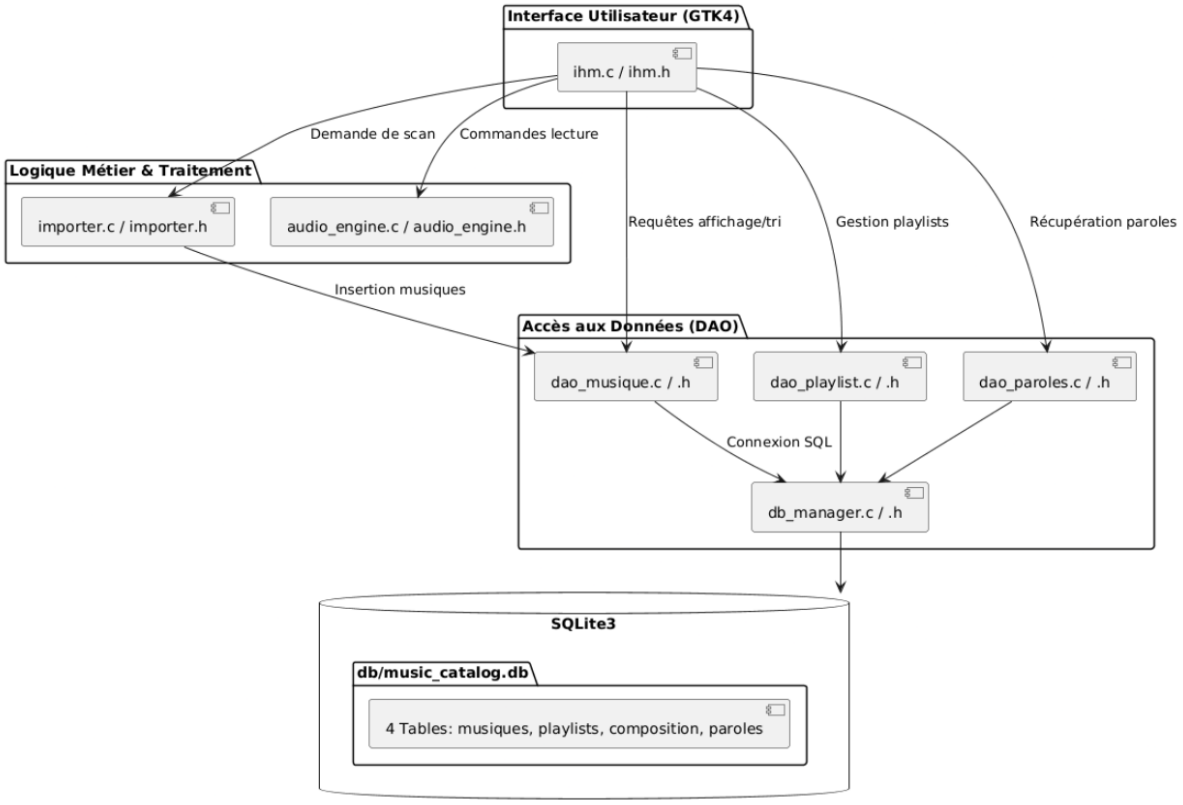


Diagramme de structure du programme Final :


Interface Utilisateur (GTK4)
ihm.c / ihm.h
Demande de scan
Commandes lecture
Logique Métier & Traitement
importer.c / importer.h
audio_engine.c / audio_engine.h
Requêtes affichage/tri
Gestion playlists
Récupération paroles
Insertion musiques
Accès aux Données (DAO)
dao_musique.c / .h
dao_playlist.c / .h
dao_paroles.c / .h
Connexion SQL
db_manager.c / .h
SQLite3
db/music_catalog.db
4 Tables: musiques, playlists, composition, paroles